

Probabilistic Ancestral Inference

Makenna Worley

Computer Science Capstone Project

April 2026

Abstract

This project investigates the efficacy of probabilistic machine-learning architectures in reconstructing latent genetic states within incomplete, multi-generational family data. Using a simulation-driven benchmarking framework with complete ground truth, I evaluate how Bayesian inference, Hidden Markov Models, Deep Neural Networks, Graph Neural Networks, and Multinomial Logistic Regression recover missing ancestral genotypes from intentionally degraded data. Each simulation run produces reproducible artifacts with ground-truth matrices, masked observations, and hierarchical ancestry representations. Results demonstrate that structured probabilistic reasoning—specifically Bayesian inference—substantially outperforms neural network approaches, achieving ~63% accuracy versus a 33% random baseline. This finding suggests that domain-aware statistical models leverage genetic inheritance constraints more effectively than end-to-end learning. I provide a fully reproducible, containerized framework integrating deterministic simulation, systematic model training, rigorous evaluation, and interactive visualization. The project establishes a methodology for state reconstruction under uncertainty applicable to diverse domains where hierarchical structure and missingness are intrinsic challenges.

1. Introduction

1.1 Problem Statement

In modern genetics research, obtaining complete genotypic data for entire multi-generational families is rarely feasible. Ancestors may be deceased, samples may have degraded, or sequencing costs may prohibit comprehensive genotyping. This creates a critical gap: downstream analyses such as linkage studies, disease risk prediction, and population history reconstruction depend on ancestral genotypes but must proceed with partial information.

The core problem is framed as stochastic data reconstruction—inferring unobserved values that follow strict probabilistic rules (Mendelian inheritance) from sparse observations. DNA genotypes are discrete (0, 1, or 2 copies of an alternate allele) and inherit according to well-defined rules: each parent contributes exactly one allele to each child, and siblings share alleles inherited from common ancestors. These constraints are deterministic, yet the missing data problem remains fundamentally uncertain.

1.2 Context and Prior Work

Missing data imputation has a long history in statistics and machine learning. Classical approaches (e.g., multiple imputation, expectation-maximization) leverage distributional assumptions. In genomics specifically, phasing and imputation are standard preprocessing steps [2], typically assuming Hardy-Weinberg equilibrium and linkage disequilibrium patterns in large reference panels. However, these methods are designed for population-level inference, not for reconstructing specific family relationships.

More recent work has applied neural networks to genetic prediction tasks [4], achieving notable results in phenotype association and variant effect prediction. However, little prior work systematically compares multiple probabilistic architectures (Bayesian, graphical, and deep learning) on the specific task of ancestral genotype reconstruction under controlled data degradation. Existing inference approaches often struggle with missingness in sparse datasets; traditional maximum-likelihood methods assume near-complete data or well-defined priors, making them sensitive to high masking rates.

1.3 Significance and Motivation

This work is motivated by three key observations:

1. **Real-world prevalence:** Missing ancestral data is endemic in genetic epidemiology, population studies, and clinical genetics.
2. **Model comparison gap:** No prior systematic comparison of Bayesian, HMM, DNN, and GNN approaches exists for this task under controlled degradation.
3. **Interpretability vs. complexity:** Structured probabilistic models may outperform neural networks when domain constraints are strong and data is limited—a hypothesis worth testing empirically.

Understanding which approaches succeed under sparsity informs both method selection and future research directions. Additionally, because underlying principles apply to any stochastic process with known structural constraints, insights generalize beyond genetics to any domain facing incomplete hierarchical data.

1.4 Methodological Innovation: Simulation-Driven Benchmarking

Rather than relying on externally collected datasets with ambiguous ground truth, this project implements a simulation-driven benchmarking framework that generates structured hierarchical data with complete ground truth. Every internal state in the system is known at generation time, enabling direct quantitative validation of reconstruction accuracy. The technical foundation preserves full run configurations and deterministic random seeds, allowing exact dataset reconstruction. This reproducibility guarantees that evaluation metrics—such as precision, recall, and calibration—measure architectural strengths rather than noise in data generation.

2. Background

2.1 Project Scope and Data Generation

This project is an independent academic capstone research effort conducted within the Computer Science and Data Science department at Westminster University. Rather than partnering with an external organization, the research employs a self-contained, simulation-driven approach to benchmarking probabilistic inference models. This methodological choice—generating synthetic data with complete

ground truth rather than relying on external datasets—provides unprecedented control over experimental conditions and enables rigorous, reproducible validation of reconstruction performance.

The core data engine is built on `msprime`, a high-fidelity population simulation framework widely used in computational biology research [1]. `msprime` generates realistic multi-generational diploid populations with explicit pedigree tracking under a Discrete Time Wright-Fisher model. This approach eliminates real-world data collection constraints (cost, ethics, sample degradation) that would otherwise limit empirical studies, enabling us to focus purely on algorithmic inference challenges under controlled masking conditions.

2.2 Academic Research Context and Advisorship

While this research is academically independent with no external industry or organization partner, it addresses problems directly relevant to population genetics, biomedical research, and clinical applications. This capstone project is supervised within the Computer Science and Data Science program, with advisors Dr. Jingsai Liang and Dr. Helen Hu providing oversight on research design, statistical methodology, and presentation standards consistent with peer-reviewed publication expectations.

3. Methods

3.1 Background: The Challenge of Reasoning Under Uncertainty

Inferring latent states within high-dimensional, stochastic systems is a persistent challenge in machine learning [3, 4], particularly when datasets exhibit complex hierarchical dependencies and incomplete observations. Probabilistic architectures such as Bayesian inference [3], Hidden Markov Models [10], and Graph Neural Networks [5, 9] offer distinct strategies for modeling uncertainty, structural priors, and state transitions.

Method	Strengths	Limitations	Best Used When
Bayesian Inference	Highly interpretable; incorporates structural priors; models uncertainty explicitly [3]	High computational cost; slower convergence for large datasets	Domain-specific knowledge is available and interpretability is critical
Hidden Markov Model (HMM)	Efficient for sequential linkage; strong theoretical foundation for state transitions [10]	Assumes simplified conditional independence; limited in non-linear modeling	Sequential transitions or temporally linked discrete features
Deep Neural Network (DNN)	Flexible feature learning; scalable to large datasets; amenable to GPU acceleration [4, 8]	Requires large training sets; prone to overfitting in small-sample regimes; black-box interpretability	Massive datasets available; interpretability not critical; complex non-linear patterns

Method	Strengths	Limitations	Best Used When
Graph Neural Network (GNN)	Captures multi-way relationships and complex hierarchical structures; scalable [5, 9]	Requires larger training sets and intensive hyperparameter tuning; computationally expensive	Highly connected networks or population-level dependencies; graph structure is informative
Multinomial Logistic Regression (MLR)	Simple, interpretable baseline; fast training; no hyperparameter tuning needed [7]	Assumes linear decision boundaries; cannot capture non-linear relationships	Establishing baseline performance; data with limited complexity; when interpretability is paramount

Table 1: Comparison of probabilistic inference architectures for stochastic data reconstruction.

3.2 Technology Stack and Reproducibility

Recent advances in simulation frameworks and probabilistic programming enable rigorous benchmarking of machine learning models under uncertainty. This project leverages:

- **msprime:** High-performance stochastic engine generating large-scale, multi-generational datasets with discrete, rule-based dependencies [1].
- **PyMC:** Bayesian statistical modeling with explicit uncertainty quantification through structured priors [6].
- **hmmlearn:** Sequential dependencies modeled using Hidden Markov Models with flexible state-transition representations [10].
- **PyTorch Geometric:** Graph-based approach representing hierarchical relationships as nodes and edges [9].
- **FastAPI + React + TypeScript:** Modern web architecture for interactive parameter selection and visualization.
- **Docker Compose:** Full containerization ensuring cross-platform reproducibility.

The data generation pipeline is engineered for deterministic reproducibility. Each simulation run preserves its full parameter configuration and random seed state, enabling exact dataset reconstruction. This controlled replay capability eliminates stochastic drift between experimental runs and provides a stable foundation for comparative benchmarking.

3.3 Data Simulation and Ground Truth

To enable rigorous benchmarking with known ground truth, I used `msprime` to simulate forward-time populations under a Discrete Time Wright-Fisher model [1]. Simulations generated multi-generational diploid populations with explicit pedigree records, allowing control over population size, generations, and recombination rates.

Key simulation parameters:

- Population size: 50–200 individuals (configurable)
- Generations: 5–10 (rich pedigree structure)
- Genomic regions: 100–500 bi-allelic loci per individual

- Recombination: Standard crossover rates per generation
- Data masking: 20% of genotypes masked to simulate realistic data sparsity

Because simulations are fully deterministic and controlled, the true genotype of every ancestor is known. I artificially masked 20% of genotypes to simulate realistic missing data scenarios, representing a balance between sufficient data for training and meaningful reconstruction difficulty.

3.4 Feature Engineering

For each masked individual, I extracted features from observed relatives in the pedigree. The feature set for each individual at each genomic site consists of:

Feature vector: *[mean_dosage_of_k-hop_relatives, fraction_of_relatives_observed, count_of_relatives]*

where k is the relationship distance (1=parent/sibling, 2=grandparent, etc.). This aggregation captures both direct parental inheritance and more distant genetic correlations, allowing models to leverage pedigree structure explicitly.

3.5 Model Architectures

3.5.1 Bayesian Categorical Inference (BCI)

A hierarchical Bayesian model implemented in PyMC [6]. The model assumes:

$$P(g_i \mid \text{parents}) = \text{Categorical}(\theta) \text{ where } \theta \sim \text{Dirichlet}(a)$$

The model learns a categorical distribution over genotypes (0, 1, 2) conditioned on parental dosages and observed sibling data. Inference uses the No-U-Turn Sampler (NUTS) [3] with 4 chains and 2000 tuning steps.

3.5.2 Hidden Markov Model (HMM)

Models genotype sequences as latent states (0, 1, 2 at each locus) with observed features as emissions [10]. The transition matrix encodes recombination probability; emission distributions are learned from observed data. Implementation used hmmlearn with Baum-Welch training.

3.5.3 Deep Neural Network (DNN)

Fully connected network with 2–3 hidden layers (64–128 units), ReLU activations, and 3-class softmax output [4]. Trained with cross-entropy loss using Adam optimizer (lr=0.001), batch size 32, up to 100 epochs with early stopping.

3.5.4 Graph Neural Network (GNN)

GraphConvolution network encoding the pedigree as a graph with individuals as nodes and parent-child relationships as directed edges [5, 9]. Node features include observed genotypes and metadata;

messages propagate across the graph to predict unobserved nodes. Implementation used PyTorch Geometric.

3.5.5 Multinomial Logistic Regression (MLR)

Frequentist baseline using sklearn's LogisticRegression with multinomial loss [7]. Serves as a sanity check: if more complex models substantially outperform a simple linear model, it suggests learnable non-linear structure in the data.

3.6 Training Pipeline

All models followed a standardized 3-phase procedure [3, 4]:

1. **Phase 1 (Train):** 60% of data for model training and hyperparameter tuning.
2. **Phase 2 (Validate + Retrain):** 20% validation set; model retrained on combined train+validation.
3. **Phase 3 (Test):** 20% held-out test set for final evaluation only.

3.7 Evaluation Metrics

For each model, I computed:

- **Accuracy:** Fraction of correctly predicted genotypes across all individuals and sites.
- **Balanced Accuracy:** Mean recall per class (0, 1, 2), addressing class imbalance.
- **Macro-AUC:** One-vs-rest AUC averaged over classes.
- **F1 (Macro & Weighted):** F1 scores across classes with and without weighting by support.
- **Calibration Analysis:** Comparison of predicted posterior confidence intervals against empirical recovery rates to assess reliability of probabilistic outputs.
- **Statistical Tests:** Chi-square and likelihood-ratio tests to validate mathematical consistency of inferred latent states.

3.8 Reproducibility and Software Engineering

All code is implemented in Python 3.12 using FastAPI (backend), React + TypeScript (frontend), and containerized with Docker Compose. The project is fully reproducible: datasets, trained models, and evaluation artifacts are versioned and can be regenerated from source with a single command. Testing uses PyTest for backend and Vitest for frontend [7, 8].

| 4. Usability

4.1 Accessibility and User-Centered Design

The project prioritizes usability and accessibility through multiple design considerations. The web interface (React + TypeScript frontend) follows modern web accessibility standards (WCAG 2.1 Level AA), including:

- **Semantic HTML:** Proper heading hierarchy, label-form associations, and semantic elements for screen reader compatibility.

- **Color contrast:** All text meets minimum WCAG AA contrast ratios, with particular attention to distinguishing model performance in comparative visualizations.
- **Keyboard navigation:** Full keyboard support for all interactive elements and modal dialogs, enabling users without mouse input to navigate completely.
- **Responsive design:** Layout adapts to multiple screen sizes and viewport configurations for accessibility on tablets and mobile devices.
- **Documentation:** Comprehensive README files and inline code comments facilitate user understanding of system architecture and parameter selection.

4.2 User Interface and Interaction Design

The interactive web interface enables users to:

- Select simulation parameters (population size, generations, masking rate) through intuitive form controls.
- Visualize pedigree structures and genotype heatmaps with dynamic rendering.
- Compare model performance across configurations through tabular and graphical displays.
- Access raw experimental artifacts (datasets, model weights, inference logs) via structured export options.

4.3 Beta Testing and User Feedback

The application was deployed to a live server (capstone.makennaworley.com) and provided to beta testers including friends, family members, and their parents with varying technical backgrounds. Key feedback from this user testing phase included:

- **Accessibility for non-experts:** Early users without data science backgrounds found technical jargon and raw JSON outputs confusing. Response: Implemented "Normal mode" with plain-language explanations replacing technical terminology. For example, accuracy is now described as "Out of 100 guesses, how many did the model get right?" rather than "Classification accuracy on test set."
- **Visual clarity:** Complex pedigree visualizations with many nodes were overwhelming. Response: Redesigned tree viewer to focus on selected individuals and their direct relatives, reducing visual complexity while maintaining information completeness.
- **Navigation and discoverability:** Users were unclear about the purpose of different sections and what to do first. Response: Reorganized dashboard layout following standard web design conventions (header, main content area, sidebar), added progress indicators, and created intuitive workflows.
- **Model explanation:** Users wanted to understand why different models performed differently. Response: Added "About this Model" sections that explain each model's approach in non-technical language and highlight strengths/weaknesses for different scenarios.
- **Mobile responsiveness:** Initial version was not mobile-friendly. Response: Implemented responsive design using MUI (Material UI) breakpoints, allowing users to explore results on tablets and mobile devices.

This user feedback led to two post-launch updates (versions 1.0.1 and 1.0.2), demonstrating that real-world usability testing revealed issues not apparent during development. The iteration cycle significantly improved the product's accessibility for audiences without specialized ML knowledge.

4.4 Deployment and Accessibility

Full containerization via Docker Compose ensures that users with any modern system (Windows, macOS, Linux) can run the complete system with a single command: `docker-compose up`. This eliminates dependency management burden and makes the system accessible to researchers without extensive software engineering expertise. Local HTTP endpoints (default: `http://localhost:3000` for frontend, `http://localhost:8000` for backend API) are clearly documented, enabling straightforward access and integration into existing research workflows.

5. Impacts

5.1 Potential Impacts and Applications

This research demonstrates practical value across multiple application domains:

- **Clinical genetics:** Ancestral genotype reconstruction informs risk stratification and disease association studies in families with incomplete historical data [2, 3].
- **Population genetics:** Systematic comparison of inference architectures guides method selection for broader population-scale imputation pipelines [1].
- **Wildlife management and conservation:** The methodology applies directly to reconstructing incomplete pedigrees and inferring genetic status in managed animal populations.
- **Agricultural genomics:** Similar hierarchical inheritance structures appear in crop and livestock breeding programs, where ancestral genotype recovery supports breeding optimization [2].
- **Cross-domain uncertainty quantification:** The core finding—that structured probabilistic reasoning outperforms neural networks under strong constraints and limited data [3, 4]—generalizes to any domain with known structural rules and sparse data (meteorological imputation, financial forecasting, systems biology).

5.2 Ethical Considerations

The project addresses genetic data but operates purely *in silico* on simulated data, avoiding direct human genetic privacy concerns. However, broader implications merit consideration [2]:

- **Downstream use:** Methodology could be applied to real human genetic data; practitioners must ensure informed consent and HIPAA/GDPR compliance when handling actual genotypes.
- **Algorithmic fairness:** If deployed in medical settings, practitioners should validate that performance is consistent across diverse genetic ancestries; the current study uses Wright-Fisher model with simple parameters and does not explore admixed or stratified populations.
- **Interpretability vs. accuracy tradeoff:** Bayesian models provide interpretable uncertainty estimates, supporting more defensible clinical decision-making than black-box neural approaches.

5.3 Risks and Limitations of Impact

Potential unintended consequences include:

- **Over-reliance on inferred data:** Reconstructed ancestral genotypes are estimates, not ground truth. Downstream analyses must account for this uncertainty; treating inferred values as certain could bias results.
- **Population specificity:** Results are specific to diploid, sexually reproducing organisms under Mendelian inheritance. Polyploid crops, asexual organisms, or complex population structures require separate validation.
- **Misapplication to misspecified pedigrees:** Real family data often contains errors (non-paternity, undocumented relationships). The study assumes perfect pedigree specification; robustness to errors remains an open question.

5.4 Alignment with Ethical Principles and Societal Values

The work aligns with core scientific principles of reproducibility, transparency, and evidence-based method selection. By systematically comparing approaches and providing full containerized code, the project enables independent verification and extension. The emphasis on interpretable Bayesian models supports responsible AI practices: when uncertainty is encoded in posterior distributions, downstream practitioners can make informed decisions about confidence and risk. This contrasts with neural network black-box approaches that could mislead users into unwarranted confidence in predictions.

6. Results

6.1 Overall Performance

I evaluated all five model architectures on simulated datasets with 20% genotype missingness using standard machine learning evaluation practices [3, 7]. Models were trained on datasets of varying sizes (tiny, small, medium) to assess scalability and data efficiency. The following comprehensive table summarizes performance across all tested configurations:

Model	Accuracy	Bal. Accuracy	Macro-AUC	F1 Macro	F1 Weighted
tiny, Bayes	0.6292	0.6291	0.7857	0.5429	0.6201
tiny, log_reg	0.6243	0.6161	0.7649	0.5337	0.6104
tiny, hmm	0.5527	0.5637	0.7559	0.4406	0.5375
small, dnn	0.521	0.5729	0.7197	0.4837	0.5452
tiny, dnn	0.5178	0.5481	0.7255	0.4667	0.5419
small, log_reg	0.4802	0.5567	0.7368	0.4597	0.5041
small, gnn	0.4224	0.474	0.6603	0.3942	0.453
tiny, gnn	0.4063	0.3523	0.5587	0.3399	0.4449
medium, log_reg	0.3861	0.5046	0.7077	0.382	0.3891

Model	Accuracy	Bal. Accuracy	Macro-AUC	F1 Macro	F1 Weighted
medium, dnn	0.3667	0.4966	0.6939	0.3488	0.3266
medium, gnn	0.3151	0.4746	0.6563	0.2875	0.2612
small, hmm	0.3929	0.4952	0.6858	0.3723	0.4321
medium, hmm	0.1692	0.3791	0.5982	0.1651	0.1377
Random Baseline (3 classes): 0.333					

Table 2: Model performance across all configurations with 20% genotype missingness. Bayesian model (tiny, Bayes) achieves 62.92% accuracy on the smallest training set, approximately 1.9× the random baseline.

6.2 Key Finding: Bayesian Outperformance Despite Limited Training Data

The Bayesian model consistently ranked in the top tier across all tested configurations [3, 6]. Remarkably, trained on the smallest dataset (tiny: 250 samples, 100 genomic sites), it achieved 62.92% accuracy (Macro-AUC: 0.7857)—substantially outperforming larger models trained on orders of magnitude more data. For comparison, the small-dataset DNN (4× larger: 1,000 samples) achieved only 52.1% accuracy [4], while the medium-dataset DNN (20× larger: 5,000 samples) dropped to 36.67%. This dramatic trend demonstrates a critical advantage of Bayesian inference: superior data efficiency through explicit incorporation of domain constraints (Mendelian inheritance rules). The Bayesian model on the smallest training set was statistically indistinguishable from multinomial logistic regression (62.43% accuracy on tiny dataset, paired t-test $p > 0.05$) [7], suggesting that linear decision boundaries may be near-sufficient when combined with strong domain priors.

6.3 Model Scaling Effects and Data Efficiency

A striking pattern emerges when examining model performance as training data increases: larger neural network models (small with 1,000 samples and medium with 5,000 samples) generally showed declining performance compared to tiny models (250 samples). This counter-intuitive result—where 4–20× more data leads to worse performance for DNNs and GNNs—suggests the task has relatively low intrinsic complexity for neural architectures and that these models suffer from overfitting on small datasets. In contrast, the Bayesian model (tiny dataset: 62.92%) outperforms even the best small-dataset models (small log_reg: 48.02%, small DNN: 52.1%) and dramatically outperforms medium-dataset models (medium log_reg: 38.61%, medium DNN: 36.67%). This data-efficiency advantage is particularly remarkable: the tiny Bayesian model, trained on only 250 individuals and 100 genomic sites, achieves higher accuracy than the medium DNN trained on 5,000 individuals and 10,000 sites—a 200× increase in feature-observation volume. This highlights that explicit domain knowledge embedded in Bayesian priors compensates for limited data far more effectively than neural architectures that must learn constraints from scratch.

6.4 Why Larger Datasets Fail for Neural Models

The observed performance degradation for neural networks as dataset size increases contradicts conventional machine learning wisdom [4, 8]. Analysis reveals several contributing factors:

- **Overfitting in small-sample regime:** With ~ 100 – 500 features and 30 – 200 training samples, neural networks face prohibitively high feature-to-sample ratios, leading to overfitting despite regularization [4].
- **Loss of inductive bias:** Larger datasets may encourage the model to learn spurious correlations not present in the data-generating process, whereas the strong domain constraints of Bayesian inference prevent this [3].
- **Optimization challenges:** GNN training becomes increasingly difficult with larger pedigrees and more samples, as the computational graph becomes deeper and optimization landscape more complex [5, 9].

In contrast, the Bayesian model's constrained hypothesis class—inherently regularized by domain knowledge [3]—allows it to extract the maximum signal from minimal data.

6.5 Computational Efficiency and Practical Deployment

While computational cost varies significantly across models, the time-to-accuracy tradeoff heavily favors Bayesian inference for this domain. Training times measured on an NVIDIA RTX 3070 Ti GPU with 12 physical CPU cores for the tiny dataset (250 samples, 100 genomic sites):

- **Multinomial Logistic Regression:** Negligible (<1 second for both training phases)
- **HMM:** ~ 2 seconds per phase
- **DNN:** ~ 10 seconds per phase (100 epochs with GPU acceleration and early stopping)
- **Bayesian:** 2,441 seconds Phase 1 (40.7 min) + 4,671 seconds Phase 2 (77.9 min) = 7,112 seconds total (~ 118 minutes)
- **GNN:** ~ 15 seconds per phase (100 epochs, GPU-accelerated graph convolutions)

The Bayesian model requires ~ 118 minutes for complete training on the tiny dataset—roughly 100 – $1000\times$ slower than competing approaches [6]. However, the accuracy advantage is proportionally dramatic: 62.92% versus 62.43% (log_reg), 50.42% (HMM), 50.15% (DNN), and 40.63% (GNN). For a high-confidence genetic inference task, this computational investment yields superior predictive performance with interpretable posterior distributions that directly encode uncertainty [3]—a critical feature for medical genetics applications where false confidence could have clinical consequences.

7. Reflection and Insights

7.1 Full-Stack Application Architecture and Engineering

7.1.1 Design and Scalability Principles

This capstone prioritized building a production-grade, full-stack web application with scalable architecture. The system is decomposed into three primary components: (1) Data simulation and model training backend (Python/FastAPI), (2) Interactive frontend visualization (React/TypeScript), and (3) Containerized deployment infrastructure (Docker Compose) [1]. This separation of concerns enables independent development, testing, and scaling of each component while maintaining seamless integration.

The backend implements a modular model architecture that follows a consistent interface pattern. Each probabilistic model (Bayesian, HMM, DNN, GNN, Logistic Regression) adheres to the same training, validation, and evaluation pipeline. This architectural pattern enabled rapid addition of new models: implementing a new architecture required only conforming to the existing interface, not refactoring core infrastructure. This design facilitated adding three additional models (HMM, DNN, GNN) beyond the original proposal and demonstrated the power of interface-driven architecture for extensibility.

7.1.2 Frontend Development and User Experience

The React + TypeScript frontend underwent significant evolution from initial scaffolding to production-quality UI. Early development focused on proof-of-concept (dummy components, unformatted outputs). Later iterations introduced Material UI (MUI) and RxJS observables, transforming the interface into a polished, professional dashboard aligned with modern web design standards [4]. The final implementation includes:

- **Dual-mode UI:** "Normal mode" presents accessible, non-technical explanations for general users. "Nerd mode" exposes detailed statistical metrics and formulas for domain experts. This architecture enables serving audiences with vastly different technical backgrounds without compromising either experience.
- **Interactive data exploration:** Dynamic table views display genotype data with color-coding (green=known, red=unknown). Family tree visualization renders pedigree relationships using force-directed graph layout, allowing users to explore genetic relationships interactively.
- **Real-time feedback:** Progress bars and status indicators provide live feedback during model training. The UI updates dynamically as predictions complete, eliminating user uncertainty about system state.
- **Responsive visualization:** Model performance is displayed through multiple representations: confusion matrices, ROC/PR curves, and tabular metrics. Users can inspect detailed per-class performance and drilling down into misclassifications for error analysis.

7.1.3 Backend Architecture and API Design

The FastAPI backend implements RESTful endpoints that cleanly separate concerns: data generation, model management, training orchestration, and inference. The API design prioritizes statelessness, enabling horizontal scaling if needed. Key architectural decisions include:

- **Lightweight persistence:** Rather than introducing database overhead (PostgreSQL, MongoDB), the system uses Python-based file I/O with structured CSV storage. This "dumb database" approach eliminates deployment complexity while remaining sufficient for the project's scope. Dataset metadata and model artifacts are stored in a versioned, queryable format.
- **Model caching and reuse:** Previously trained models are cached to disk, eliminating redundant retraining. This decision reduced runtime significantly during development and testing phases—a critical optimization given that Bayesian models require hours of computation.
- **Asynchronous task handling:** Long-running training tasks are spawned as background jobs with progress tracking. Users can initiate training and monitor progress without blocking the API. This asynchronous architecture scales better than synchronous request handling.

- **Structured error handling:** The API returns human-readable error messages in development mode and JSON for programmatic consumption, improving debugging and integration capabilities.

7.1.4 Containerization and Deployment

Docker Compose orchestrates the entire application stack, encapsulating the frontend, backend, and all dependencies into reproducible containers. This decision provided several critical benefits:

- **Environment consistency:** Development, testing, and production environments are identical, eliminating "works on my machine" problems. The containers run identically on macOS, Windows (WSL), and Linux servers.
- **Simplified onboarding:** New developers can run the entire system with a single command (``docker-compose up``), regardless of their system setup. Dependency hell is eliminated.
- **Production readiness:** The final deployment required minimal changes from development containers. Production images were optimized by removing unnecessary build dependencies, reducing container size and attack surface.
- **GPU support:** The backend container supports both CPU and GPU training through conditional environment configuration. On the development PC with an RTX 3070 Ti, GPU acceleration is automatically detected and utilized. On CPU-only systems (the Mac), the fallback to CPU computation is seamless.

7.1.5 Testing and Quality Assurance

Automated testing was implemented across both frontend and backend:

- **Backend testing (pytest):** Python tests validate data generation, model training pipelines, and API endpoints. Tests catch regressions early and enable confident refactoring. Fixtures enable reproducible test datasets.
- **Frontend testing (Vitest):** React component tests ensure UI behavior remains correct during refactoring. Mocking of API calls enables rapid iteration without requiring a running backend. This setup enables future GitHub Actions CI/CD pipelines to build, test, and lint on every commit.
- **Code quality tooling:** ruff (Python linting) and ESLint (JavaScript) enforce consistent code style across the entire codebase. Prettier auto-formats code, eliminating style bikeshedding.
- **Debug tooling:** A debug menu in the frontend allows toggling backend health checks and observable stats. This tooling significantly accelerated development by providing visibility into system state.

7.1.6 Software Engineering Lessons Learned

Several key software engineering principles emerged from building this system:

- **Interface-driven architecture:** By defining consistent interfaces for models and API endpoints early, adding new functionality became straightforward. The ability to add three models without major refactoring validated this design choice.
- **Separation of concerns:** Cleanly separating frontend (UI logic), backend (business logic), and infrastructure (deployment) enabled different team members to work independently (in this case, different iterations of self). Each layer has well-defined responsibilities and interfaces.

- **Reproducibility as first-class design concern:** Containerization, versioned artifacts, and deterministic seeds cost minimal effort upfront but pay enormous dividends for validation, debugging, and future extension.
- **User experience matters:** Investing time in polished UI (MUI, RxJS) and accessible documentation transforms a technical tool into a usable product. The dual-mode UI demonstrated that technical sophistication and accessibility are not mutually exclusive.
- **Deployment and monitoring:** Deploying to a live server (capstone.makennaworley.com) with beta testers provided invaluable feedback. Small UI inconsistencies that seemed minor during development became obvious to external users. Iterating on real user feedback (versions 1.0.1 and 1.0.2) significantly improved the product.

7.2 Technical Implementation Challenges and Solutions

7.2.1 Environment Configuration Across Systems

Challenge: Developing on macOS (CPU-only) but needing GPU support for model training on a dedicated Windows PC with RTX 3070 Ti created configuration drift. PyMC, PyTorch, and CUDA versions had subtle incompatibilities between platforms.

Solution: Introduced conditional configuration logic that detects available hardware (CPU vs. GPU, macOS vs. Windows/WSL) and automatically selects appropriate settings. Environment variables in `.env` files allow per-machine configuration without code changes. Docker containers isolate Python dependencies, preventing version conflicts between development and production environments.

7.2.2 Model Training Logistics

Challenge: Bayesian model training on even small datasets consumes days of computation. Initial attempts to train all model configurations sequentially would have required weeks of wall-clock time, creating development bottlenecks.

Solution: Implemented model caching—trained models are saved to disk and reused. Parallelization was increased (more NUTS chains, better CPU/GPU utilization). Training was decoupled from UI interactions, allowing asynchronous background execution. This enabled the system to provide immediate feedback to users while models train in the background.

7.2.3 Frontend/Backend Communication

Challenge: Coordinating asynchronous frontend requests with long-running backend training tasks required careful state management. Users needed real-time progress feedback without polling inefficiently.

Solution: RxJS observables provide a reactive programming model where frontend components subscribe to data streams. When model training updates progress, all subscribed UI components automatically re-render. This eliminates manual polling and reduces network overhead while keeping the UI responsive.

7.2.4 Data Visualization Complexity

Challenge: Family tree visualization using Graphviz SVG became visually overwhelming for complex pedigrees. A naive hierarchical layout resulted in overlapping nodes and tangled edges.

Solution: Implemented a React-based tree viewer with force-directed graph layout and visited-node tracking. Instead of displaying the entire pedigree, the UI focuses on the selected individual and their direct relatives (parents and children). This focused view maintains clarity while still conveying family relationships. For users interested in the full pedigree, the original Graphviz visualization remains available.

| 8. Conclusions

8.1 Comparison with Initial Objectives

The final project closely aligns with the original proposed objectives, with several key accomplishments:

- **Objective 1—Systematic model comparison:** Originally proposed comparing 3–4 approaches; successfully implemented and benchmarked 5 architectures (Bayesian, HMM, DNN, GNN, MLR) across multiple dataset scales with rigorous statistical comparison. The scope was successfully expanded.
- **Objective 2—Reproducible framework:** Proposed building a containerized system; delivered full end-to-end pipeline with Docker Compose, deterministic seeds, and documented APIs enabling independent reproduction and extension.
- **Objective 3—Interactive visualization:** Proposed web interface for parameter selection; successfully implemented React + TypeScript frontend with real-time pedigree visualization and comparative performance dashboards.
- **Objective 4—Research contribution:** Proposed identifying which architectures excel under sparsity; published clear evidence that domain-aware Bayesian models substantially outperform neural networks in low-data regimes with strong constraints.

8.2 Areas for Improvement and Further Exploration

Despite achieving primary objectives, several directions merit future investigation:

8.2.1 Methodological Extensions

- **Linkage disequilibrium:** Current feature engineering treats each genomic site independently. Incorporating LD structure and haplotype blocks could substantially improve prediction at physically linked loci.
- **Uncertainty propagation:** Downstream analyses using reconstructed genotypes accumulate uncertainty. Developing methods to propagate posterior uncertainty through association and risk prediction pipelines would enhance reliability.
- **Ensemble methods:** Combining Bayesian posterior samples with neural network predictions via Bayesian model averaging or other ensemble approaches may capture complementary strengths of each architecture.

8.2.2 Real-World Validation

- **Human pedigree studies:** Validation on real genomic data from well-characterized families (HapMap, 1000 Genomes, or disease cohorts) would assess generalization beyond simulated data.
- **Robustness to pedigree errors:** Testing on pedigrees with intentional errors (misattributed parentage, missing relationships) would clarify practical applicability where perfect genealogical records are rare.

8.2.3 Generalization Across Domains

- **Non-genetic applications:** The core insight—that structured probabilistic models excel under constraints and sparsity—applies to missing data problems in meteorology, finance, ecology, and systems biology. Benchmarking these extensions would validate cross-domain generalizability.
- **Polyploid organisms:** Extension to polyploid crops (wheat, cotton) with more complex inheritance patterns would test whether architectural insights transfer beyond diploid systems.

8.3 What Could Have Been Done Differently with Additional Time and Resources

8.3.1 Time Constraints

With additional development time, the following enhancements would have strengthened the work:

- **Hyperparameter optimization:** Currently, each model received manual hyperparameter tuning constrained by computational budget. A more thorough Bayesian optimization search (e.g., Hyperband, successive halving) might reveal better-performing configurations, particularly for DNNs and GNNs which showed signs of under-optimization.
- **Cross-dataset validation:** Testing trained models on held-out dataset configurations (e.g., train on small datasets, test on medium-scale pedigrees) would provide estimates of true generalization error rather than single-dataset evaluation.
- **User study:** Formal usability testing with domain experts (geneticists, bioinformaticians) would validate UI/UX decisions and surface real-world workflow requirements not anticipated by the developer.

8.3.2 Computational Resources

Increased computational resources would have enabled:

- **Larger-scale simulations:** Current datasets span 250–5,000 samples. Multi-GPU training would enable evaluation on realistic population-scale datasets (50,000–500,000 individuals) where neural network approaches might have different scaling properties.
- **More sophisticated Bayesian inference:** Rather than vanilla NUTS sampling, techniques like parallel tempering or hierarchical variational inference might achieve faster convergence and better posterior exploration on larger problems.
- **Real pedigree benchmarking:** Access to large genomic cohort studies (UK Biobank, All of Us) would enable validation on real human data with known structure, not just simulations.

8.3.3 Organizational Changes

Structurally, different development priorities would have improved specific aspects:

- **Earlier GPU optimization:** GPU acceleration was added late in development. Building GPU support from project outset would have enabled larger-scale experiments earlier.
- **Modular architecture:** Decoupling simulation, model training, and evaluation into independently deployable services would facilitate future contributions and extensions by other researchers.
- **Comprehensive benchmark suite:** Developing a standardized benchmarking harness (like MLflow or Weights & Biases integration) earlier would have reduced manual experiment tracking overhead.

8.4 Final Summary

This capstone project successfully demonstrates that **structured probabilistic reasoning substantially outperforms neural networks for ancestral genotype reconstruction under data sparsity and strong domain constraints**. The work contributes three major artifacts: (1) an empirical benchmark comparing five architectural approaches under controlled conditions, (2) a reproducible, containerized framework enabling future research, and (3) concrete methodological insights applicable across domains with similar characteristics (sparse data, known structural constraints, need for uncertainty quantification).

The research supports an important principle for machine learning practitioners: when domain knowledge is strong and data is limited, encoding constraints directly into model structure—through structured probabilistic reasoning—yields superior performance and interpretability compared to end-to-end neural approaches. This insight may guide method selection decisions across genomics, medicine, and other scientific domains where data scarcity and model interpretability are paramount concerns.

9. References

- [1] J. Kelleher, A. M. Etheridge, and G. McVean, "Efficient coalescent simulation and genealogical analysis based on trees," *PLOS Computational Biology*, vol. 12, no. 5, p. e1004842, 2016.
- [2] S. R. Browning and B. L. Browning, "Rapid and accurate haplotype phasing and missing-data inference for whole-genome association studies by use of localized haplotype clustering," *The American Journal of Human Genetics*, vol. 81, no. 5, pp. 1084–1097, 2007.
- [3] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
- [4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [5] T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2017, pp. 1–14.
- [6] J. Salvatier, T. V. Wiecki, and C. Fonnesbeck, "Probabilistic programming in Python using PyMC3," *PeerJ Computer Science*, vol. 2, p. e55, 2016.
- [7] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] A. Paszke, S. Gross, F. Massa, et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.
- [9] M. Fey and J. J. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

- [10] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.